Experiment 3

Working with Gradle: Setting Up a Gradle Project, Understanding Build Scripts (Groovy and Kotlin DSL), Dependency Management and Task Automation

1️⃣ Introduction to Gradle

Gradle is a powerful build automation tool used for **Java, Kotlin, and Android projects**. It provides **fast builds, dependency management, and flexibility** using Groovy/Kotlin-based scripts.

🔹 Why Use Gradle?

1. **Incremental builds** (faster execution)
2. **Dependency management** (supports Maven/Ivy)
3. **Customizable tasks**
4. **Multi-project support**

2️⃣ Installing and Setting Up Gradle in IntelliJ IDEA

Step 1: Install Gradle

Step 2: Verify Installation

gradle -v

3️⃣ Creating a Gradle Project in IntelliJ IDEA

Step 1: Open IntelliJ IDEA and Create a New Project

1. Click on **"New Project"**.

2. Select **"Gradle"** (under Java/Kotlin).

3. Choose **Groovy or Kotlin DSL (Domain Specific Language)** for the build script.

4. Set the **Group ID** (e.g., com.example).

5. Click **Finish**.

4️⃣ Build and Run a Simple Java Application

Step 1: Modify build.gradle **(Groovy DSL)**

```
plugins {
id 'application'
}

repositories {
mavenCentral()
```

```
}

dependencies {
testImplementation 'org.junit.jupiter:junit-jupiter:5.8.1'
}

application {
mainClass = 'com.example.Main'
}
```

Step 2: Create Main.java **in** src/main/java/com/example

```
package com.example;

public class Main {
public static void main(String[] args) {
System.out.println("Hello from Gradle!");
}
}
```

Step 3: Build and Run the Project

In IntelliJ IDEA, open the **Gradle tool window** (View → Tool Windows → Gradle).
Click Tasks > application > run .
Or run from terminal:

```
gradle run
```

5️⃣ Hosting a Static Website on GitHub Pages

Step 1: Create a /docs **Directory**

Create docs inside the **root folder** (not in src).
Add your **HTML, CSS, and images** inside /docs .

Step 2: Modify build.gradle **to Copy Website Files (This is optional)**

```
task copyWebsite(type: Copy) {
from 'src/main/resources/website'
into 'docs'
}
```

Step 3: Commit and Push to GitHub

```
git add .
git commit -m "Deploy website using Gradle"
git push origin main
```

Step 4: Enable GitHub Pages

Go to **GitHub Repo → Settings → Pages**.

Select the /docs **folder** as the source.
Your website will be hosted at:
https://yourusername.github.io/repository-name/

6️⃣ Testing the Website using Selenium & TestNG in IntelliJ IDEA

Step 1: Add Selenium & TestNG Dependencies in build.gradle

```
dependencies {
testImplementation 'org.seleniumhq.selenium:selenium-java:4.28.1' // use the latest stable version
testImplementation 'org.testng:testng:7.4.0' // use the latest stable version
}
test {
useTestNG()
}
```

Step 2: Write a Test Script (src/test/java/org/test/WebpageTest.java **)**

```
package org.test;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.Assert;
import org.testng.annotations.AfterTest;
import org.testng.annotations.BeforeTest;
import org.testng.annotations.Test;

import static org.testng.Assert.assertTrue;

public class WebpageTest {
private static WebDriver driver;

@BeforeTest
public void openBrowser() throws InterruptedException {
driver = new ChromeDriver();
driver.manage().window().maximize();
Thread.sleep(2000);
driver.get("https://sauravsarkar-codersarcade.github.io/CA-GRADLE/");
 }

@Test
public void titleValidationTest(){
String actualTitle = driver.getTitle();
String expectedTitle = "Tripillar Solutions";
Assert.assertEquals(actualTitle, expectedTitle);
```

```
assertTrue(true, "Title should contain 'Tripillar'");

}

@AfterTest
public void closeBrowser() throws InterruptedException {
Thread.sleep(1000);
driver.quit();
}


}
```

Step 3: Run the Tests
Open the **Gradle tool window** in IntelliJ.
Click Tasks > verification > test . **“Recommended“**
Or run from terminal:

```
Gradle test                // Fails sometimes due to terminal issues
```

7️ Packaging a Gradle Project as a JAR

Step 1: Modify build.gradle **for JAR Packaging**

```
plugins {
id 'java'
id 'application'
}

application {
mainClass = 'com.example.Main'
}
jar {
manifest {
attributes 'Main-Class': 'com.example.Main' // This tells Java where to start execution
}
}
```

Step 2: Build and Package the JAR

```
gradle jar
```

Step 3: Run the JAR

```
java -jar build/libs/<my-gradle-project>.jar
```